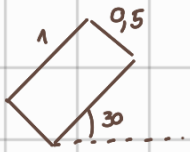
EXERCIȚII - EGC

① $\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = M \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$

Obiect inițial



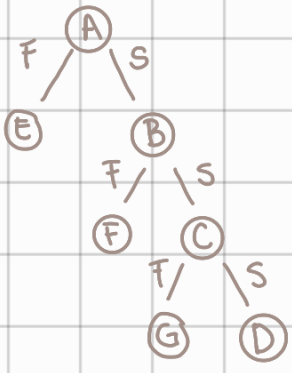
$$\Rightarrow \begin{bmatrix} 1 & 0 & 0.7 \\ 0 & 1 & 0.3 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0.5 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos 30^\circ & -\sin 30^\circ & 0 \\ \sin 30^\circ & \cos 30^\circ & 0 \\ 0 & 0 & 1 \end{bmatrix} \Rightarrow \begin{bmatrix} 1 & 0 & 0.7 \\ 0 & 0.5 & 0.3 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \frac{\sqrt{3}}{2} & -\frac{1}{2} & 0 \\ \frac{1}{2} & \frac{\sqrt{3}}{2} & 0 \\ 0 & 0 & 1 \end{bmatrix} \Rightarrow$$

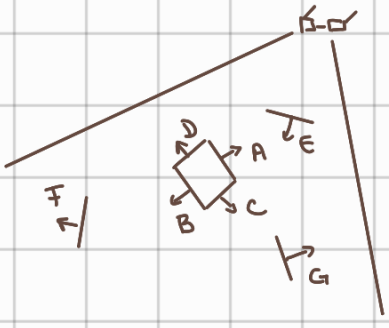
translație · scalare · rotație 30°

$$\Rightarrow \begin{bmatrix} \sqrt{3}/2 & -1/2 & 0.7 \\ 1/4 & \sqrt{3}/4 & 0.3 \\ 0 & 0 & 1 \end{bmatrix}$$

② ALG BSP

a) ordine A, B, C, D, E, F, G





b) ordinea „din față în spate"

③ Volume de umbră - formate din sursă de lumină și un poligon generator

PUV - poligoane de umbră vizibile din poz. observatorului

PUM - poligoane de umbră care nu sunt vizibile din poziția observatorului

} se folosesc pt. det. umbririi produse de poligonul generator în scenă

Clasificarea PUV și PUM se poate face pe baza normalelor la poligoane și a poz. observatorului. Fie VP vectorul din poz. observatorului (V) în punctul P. Se asociază poligoanelor de tip PUV val. +1, iar celor de tip PUM val -1. Când VP traversează un poligon de umbră, se adună la contor val. asociată poligonului. Punctul P este umbrit dacă val contorului este pozitivă în P.

④ Fereastră - poartă

- fereastră : (0,0) - (1,1)
- poartă : (0,0) - (400,200)
- P(0.7, 0.8), scalare uniformă și centrare în poartă)

$s = \min(s_x, s_y)$

$$s_x = \frac{x_{pmax} - x_{pmin}}{x_{fmax} - x_{fmin}} = \frac{400 - 0}{1 - 0} = 400$$

$s_y = 200 \Rightarrow s = 200$

$$t_{sx} = \frac{x_{pmax} - x_{pmin} - s * (x_{fmax} - x_{fmin})}{2}$$

$$t_{sx} = \frac{400 - 0 - 200\,(1 - 0)}{2} = \frac{200}{2} = 100$$

$$t_{sy} = \frac{200 - 0 - 200\,(1 - 0)}{2} = 0$$

$tx = xpmin - Sx * xfmin = 0 \Rightarrow tx + tsx = 100$

$ty = 0 \Rightarrow ty + tsy = 0$

$$\begin{cases} xp = xf * s + tx + tsx \Rightarrow xp = 0.7 \cdot 200 + 100 = 240 \\ yp = yf * s + ty + tsy \Rightarrow yp = 0.8 \cdot 200 + 0 = 160 \end{cases}$$

⑤ $\begin{bmatrix} -1 & 0 & 3 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{bmatrix}$ $\Rightarrow$ avem o scalare $\begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$, iar apoi o translație $\begin{bmatrix} 1 & 0 & 3 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{bmatrix}$

⑥ Fereastră - poartă

- fereastră - pătrat (0,0 — 1,1)
- poartă - dreptunghi (0,0 — 200,100)
- P(0,1), scalare uniformă și centrare în poartă)

$s = \min(sx, sy)$

$sx = \frac{xpmax - xpmin}{xfmax - xfmin} = 200$ $\Rightarrow s = 100$

$sy = 100$

$tsx = \frac{xpmax - xpmin - s(xfmax - xfmin)}{2}$

$tsx = \frac{200 - 100}{2} = 50$

$tsy = \frac{100 - 100}{2} = 0$

$tx = xpmin - Sx \cdot xfmin = 0$

$ty = 0$

$$\Rightarrow \begin{cases} xp = xf * s + tx + tsx = 0 + 50 = 50 \\ yp = 100 \end{cases}$$

⑦ Descriere Alg. Z-Buffer - alg. de elim. a fragmentelor nevizibile ale primitivelor grafice. Este executat în GPU, fiind integrat în procesul de rasterizare a primitivelor.

Logică: Dacă 2 fragmente aflate pe același proiector, rezultate din rasterizarea a două primitive diferite, au coordonata z diferită: fragmentul având coordonata z mai mică va fi afișat în pixelul (x,y)

⑧ Descriere Alg. BSP - intrare: lista primitivelor ce compun scena, arborele BSP este indep. de poz. observatorului.

- Fiecărui nod îi corespunde un plan de partiționare, care se împarte în 2 subspații < față / spate
fiecare generând un subarbore.
- Afișarea ține cont de poz. observatorului < front to back / back to front

⑨ Descriere Alg. Pictorului - alg. din categoria hidden surface removal

- intrare: lista poligoanelor care alc. scena 3D
- ideea: eliminarea părților nevizibile se rezolvă prin afișare "din spate în față", la fel ca în pictură
- nu e necesar testul de adâncime, eficient când poligoanele sunt disjuncte pe z

⑩ Introducerea umbrelor în Alg. 2-buffer - shadow-mapping

- metodă de generare a umbrelor bazată pe rasterizarea de 2 ori a scenei.

° în prima etapă se construiește buffer-ul z al scenei văzută din poz. sursei de lumină, notat cu zs. El conține coordonatele z ale fragmentelor care primesc lumină de la sursă

° în a doua etapă se construiește imaginea văzută din poz. observatorului, utilizând buffer-ul zs pt. det. punctelor aflate în umbră.

↗ obMg VobsPdec ÎPdisp. VIEW e

⑪ Împărțirea Perspectivă - se aplică coor. de decupare în urma proiecției, rezultând coor. dispozitiv normalizate

- se trece de la coord. omogene la coord. carteziene

- vârfurile rezultate prin Î.P. sunt asamblate în primitive, care sunt decupate la frontiera vol. viz. canonic

⑫ Fereastră-poartă
- fereastră: (0,0) - (1,1)
- poartă: (0,0) - (400,200)
P: (0.5, 1)

$$S_x = \frac{x_{pmax} - x_{pmin}}{x_{fmax} - x_{fmin}} = 400,\ S_y = 200$$
$$t_x = x_{pmin} - S_x * x_{fmin} = 0,\ t_y = 0$$
$$\Rightarrow \begin{cases} x_p = x_f * S_x + t_x = 0.5 \cdot 400 = 200 \\ y_p = y_f * S_y + t_y = 1 \cdot 200 = 200 \end{cases}$$

⑬ Tipuri de umbră → umbră proprie: generată de obiectul însuși, care împiedică lumina să ajungă la unele din fețele sale: fețele umbrite de umbra proprie sunt fețele auto-obturate atunci când scena este văzută din poziția sursei de lumină

→ umbră purtată: umbra pe care un ob. o produce pe alte părți ale scenei, la care lumina nu ajunge din cauza obiectului

⑭ Alg. Bresenham - generalizat - $P_1(12,7)$ și $P_2(5,20)$

a) din ce octant face parte vectorul P_1P_2?

$dx = x_2 - x_1 = 5 - 12 = -7 < 0$
$dy = y_2 - y_1 = 20 - 7 = 13 > 0$
$abs(dx) < dy$
$\Rightarrow$ octantul 3

octantul 1: $dx>0, dy>0, dx \geq dy$
octantul 2: $dx>0, dy>0, dx < dy$
octantul 3: $dx<0, dy>0, abs(dx) < dy$
octantul 4: $dx<0, dy>0; abs(dx) \geq dy$
octantul 5: $dx<0, dy<0; abs(dx) \geq abs(dy)$
octantul 6: $dx<0, dy<0; abs(dx) < abs(dy)$
octantul 7: $dx>0, dy<0; dx < abs(dy)$
octantul 8: $dx>0, dy<0, dx \geq abs(dy)$

b) Dacă avem un pixel (x_i, y_i) care rez. în urma rasterizării vectorului, care ar putea fi următorul pixel rasterizat (deplasare orizontal / diagonal)?

$dy > abs(dx) \Rightarrow y$ se modif. în sensul săgeții octantului $\Rightarrow y+1$

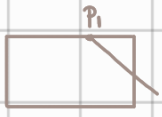
avem octant 3 $\Rightarrow x$ este x sau $x-1$

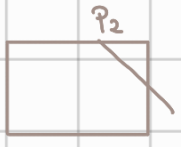
$\Rightarrow (x, y+1)$ și $(x-1, y+1)$

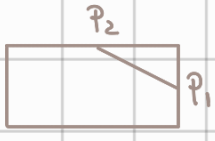
(15) Alg. Cohen-Sutherland - care sunt pașii?

1001	1000	1010
0001	0000	0010
0101	0100	0110

$\Rightarrow$
- $P_1 : 1000 = cod1 \; != 0$
- $P_2 : 0010 = cod2 \; != 0$
- $cod1 \; \& \; cod2 == 0$
- verificăm P_1 e dif de 0 ✓
- P_1 este deasupra $\Rightarrow$ interschimbăm $y_1 = ymin$

$\Rightarrow$ 

- $P_1 = 0000$ și $P_2 = 0010$ (nu sunt îndeplinite cond)
- verificăm $P_1 \Rightarrow$ e 0 se interschimbă cu P_2 $\Rightarrow$ 

- $P_2 = 0000$ și $P_1 = 0010$ (nu sunt îndep cond)
- verificăm P_1 e dif de 0 ✓
- P_1 în dreapta $\Rightarrow$ interschimbăm $x_1 = x\,max$ $\Rightarrow$ 

- $P_1 = 0000$ și $P_2 = 0000$ $\Rightarrow$ ACCEPTAT

Algoritm - $cod1 == 0$ && $cod2 == 0$ ACCEPTAT

- $cod1$ și $cod2 \; != 0$ REJECTAT
- verificăm în ce parte e P_1
  - dacă $cod1 = 0 \Rightarrow$ interschimbăm P_1 cu P_2
  - P_1 în stânga $\Rightarrow x_1 = x\,min$
  - P_1 în dreapta $\Rightarrow x_1 = x\,max$
  - P_1 sub $\Rightarrow y_1 = y\,max$
  - P_1 deasupra $\Rightarrow y_1 = y\,min$
- calc punctul de intersecție

(se întoarcem)

(16) Maparea inversă

rezoluție imagine : 1024×1024 ; $(u, v) - (0.6, 0.7)$

a) texelii la GL_NEAREST ?

$0.6 \cdot 1024 = 614.4$

$0.7 \cdot 1024 = 716.8 \Rightarrow 1024 - 716.8 = 307.2$ (se rotunjește) $\Rightarrow (614, 307)$

b) texelii la GL_LINEAR?

$(614, 307), (614, 308), (615, 307), (615, 308)$

- se calc pt. fiecare fragment
- calc. adresa (uf, vf) pe care o trimite la fragment shader (se obțin prin interpolarea coordonatelor (u_i, v_i))
- în fragment shader se obține o cul. din textură, folosindu-se (uf, vf) ← intrare în fg. shader

GL_NEAREST → considerăm culoarea texelului cu adresa cea mai apropiată

→ rapid, produce defecte de imagine

GL_LINEAR → efectuăm o interpolare biliniară între culorile celor mai apropiați texeli

→ scade viteza de calcul a culorii, produce un efect de încețoșare

- numai pt. suprafețele 3D care pot fi definite într-un spațiu parametric (t_1, t_2)
- corespondența se stabilește între spațiul (t_1, t_2) și (u, v)

(17) Fereastră - poartă

$$\begin{cases} \text{fereastră} : (0,0) - (1,1) \\ \text{poartă} : (0,0) - 1400,200) \\ \text{Punct} : (0.7, 0.8) - \text{scalare uniformă și centrare în poartă.} \end{cases}$$

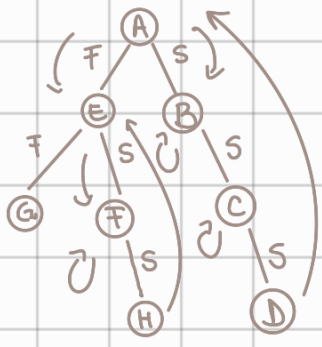
$$s_x = \frac{x_{pmax} - x_{pmin}}{x_{fmax} - x_{fmin}} = 400 \quad \Big| \Rightarrow s = \min(s_x, s_y)$$
$$s_y = 200 \qquad s = 200$$

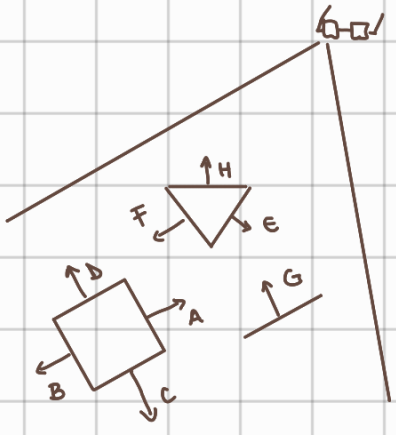
$$t_{sx} = \frac{x_{pmax} - x_{pmin} - s(x_{fmax} - x_{fmin})}{2}$$
$$t_{sx} = \frac{400 - 200}{2} = 100$$
$$t_{sy} = 0$$

$$t_x = x_{pmin} - s_x \cdot x_{fmin}$$
$$t_x = 0$$
$$t_y = 0$$

$$\begin{cases} x_p = x_p * s + t_x + t_{sx} \\ x_p = 0.7 \cdot 200 + 100 \\ x_p = 240 \\ y_p = 160 \end{cases}$$

(18) BSP a) ordine : A, B, C, D, E, F, G, H





b) **afișare back-to-front** : B, C, D, A, F, H, E, G

Pornim de la rădăcină: A - observatorul se află în F lui A, deci luăm ramura din S a lui A.

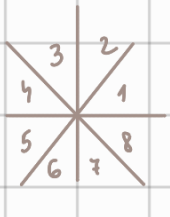
Observatorul în S lui B ⇒ luăm F lui B, nu avem nimic ne întoarcem la B, îl adăugăm la parcurgere

La C, nu e în câmpul vizual ⇒ mergem pe F, nu avem nimic, ne întoarcem la C și adaug ...

(19) Alg Bresenham : $P_1(7,10)$, $P_2(20,5)$

a) din ce octant face parte vectorul P_1P_2? Cum s-a det octantul?

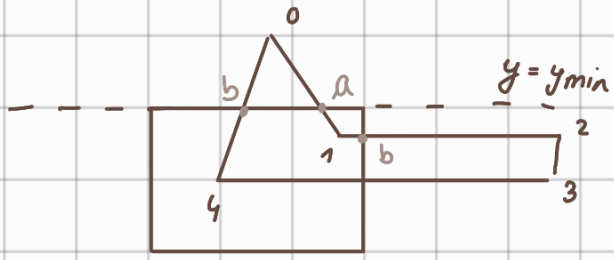
$$\begin{cases} dx = x_2 - x_1 \\ dy = y_2 - y_1 \end{cases} \Rightarrow \begin{cases} dx = 20 - 7 = 13 > 0 \\ dy = 5 - 10 = -5 < 0 \\ dx \geq abs(dy) \end{cases} \Rightarrow \text{octantul 8}$$



b) adresa urm. pas?

dx mai mare ⇒ x+1 } ⇒ (x+1, y), (x+1, y-1)

suntem în octantul 8 ⇒ y sau y-1



(20) Alg Sutherland-Hodgman

- poligon laturile: [0,1), [1,2), [2,3), [3,4), [4,0)
- poligonul rezultat în etapa decupării față de $y = y_{min}$

Algoritm - decupare poligoane complexe față de o zonă dreptunghiulară

1. se porneste cu un poligon de tăiere și unul de intrare închise și convexe
2. pt. fiecare latură a poligonului de intrare
   ① verificăm intersecția dintre latură și poligonul de tăiere
   ② dacă se intersectează, calculăm pct. de intersecție
   ③ se adaugă la lista rez.
3. se cont. parcurgerea tuturor laturilor
4. avem rez. final.
5. eliminăm vârfurile redundante

* întâmpină probleme pt poligoane concave

0-1: a,1 → a,1

1-2: 1,2 → a,1,1,2

2-3: 2,3 → a,1,1,2,2,3

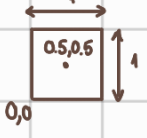
3-4: 3,4 → a,1,1,2,2,3,3,4

4-0: 4,b → a,1,1,2,2,3,3,4,4,b

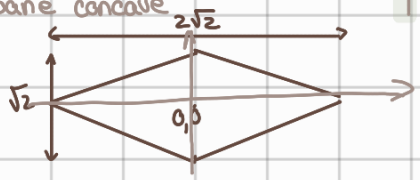
Lista finală: a,1,2,3,4,b

(21)

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = M \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



Obiect inițial



Obiect transformat

$M = S \cdot R \cdot T$

(22) **Modelul Gouraud** - fiecărui vârf al poligonului îi este asociată o culoare

- culorile fragmentelor se obțin prin interpolarea liniară a cul. din vârfuri
- cul. din vârf - se calc aplicând modelul de ilum. locală (în verdex shader), cul. fiecărui frag. calc. în GPU de procesul de rasterizare
- nu permite calc. luminii reflectate specular
- se pot aproxima corect în int. triunghiului: comp. speculară și factorul de atenuare

(23) **Modelul Phong** - fiecărui vârf îi este asociată normala vârfului

- normala și poziția vârfui (coon. globale) sunt ieșiri din vertex shader
- cul. fragmentului se calc. în fragment shader aplicând modelul de iluminare locală
- normala este normalizată în fragment shader
- permite redarea reflexiei speculare în orice fragment al unui poligon

(24) **Modelul Lambert** - toate frag. unui poligon sunt afișate cu aceeași culoare, calculată folosind modelul de iluminare locală

(25) **Modelul RGB** - orientat către echipamente

Roșu: 1 0 0, Verde: 0 1 0, Albastru: 0, 0, 1

Cian: 0, 1, 1 - cul. complementară lui Roșu

Pe diagonala Alb - Negru - nuanțe de gri obținute în prop. egale de R, G, B

Transformare liniară

Modelul CMY

Cian: 1, 0, 0; Magenda: 0, 1, 0; Galben: 0, 0, 1

Roșu: 0, 1, 1 - complementară lui Cian

Modelul HSV - orientat către utilizator

- $0 \leq V \leq 1$, $0 \leq S \leq 1$, $0 \leq H \leq 360$
- cul. complementare la 180° pe baza hexagonului
- saturația = dis (axa hexagonului, punct)
- alb: $S=0, V=1$, gri: $S=0, 0 \leq V \leq 1$
- cul. pure: $S=1, V=1$

Modelul HSL

- cul. primare cu saturație maximă și complementarele lor sunt reprez prin $S=1, L=0.5$

(26) Interpolarea în spațiul culorilor

Fie 2 cul. specificate în RGB: $C_1 = (1,0,0)$ și $C_2 = (0,1,0)$. Le interpolăm cu ponderi egale în modelele RGB și HSV.

→ în modelul RGB

$C = t * C_2 + (1-t) * C_1$, $t = 0.5$ (ponderi egale)

$C = (0.5, 0.5, 1)$ → în HSV : $(60, 1, 0.5)$

→ în modelul HSV

C_1 în HSV $\Rightarrow (0,1,1)$

C_2 în HSV $\Rightarrow (120,1,1)$

$\Rightarrow C = 0.5 \cdot (0,1,1) + 0.5\,(120,1,1) = (60,1,1)$

Explicație pt HSV

$\max(R,G,B) = 0.5$, $\min(R,G,B) = 1$

$V = \max = 0.5$

$S = \frac{\max - \min}{\max} = 1$

$$H = \begin{cases} \frac{G-B}{\max - \min}, & \text{dacă } R = \max \\ \frac{B-R}{\max - \min} + 2, & \text{dacă } G = \max \\ \frac{R-G}{\max - \min} + 4, & \text{dacă } B = \max \end{cases}$$

$\Rightarrow H = \frac{0,5}{0,5} \cdot 60 = 60$

pt. HSL

$L = \frac{\max - \min}{2}$

$S = \frac{\max - \min}{\max + \min}$ dacă $L \leq 0.5$

SAU

$S = \frac{\max - \min}{2 - \max - \min}$ dacă $L > 0.5$

$\times 60$

(27) Alg. Ray-Tracing

- imaginea nu se obține prin rasterizarea primitivelor, ci prin intersecția scenei cu raze care pleacă din poz. observatorului
- depinde de direcția razei primare
- se pot reda umbre și reflexii ale luminii
- P - rază primară ce trece pixel și intersectează scena 3D, T - raza transmisă calc. cu Snell
- L - raza către sursa de lumină; R - raza reflectată specular, simetrică față de N cu L

(28) Coord. de textură $(i,j) = (-0.3, 3.2)$. La mom. eșantionării texturii, în funcție de prop. de filtrare, cul. obținută poate fi:

$(u,v) = (u_f - [u_f], v_f - [v_f])$ $\quad (u,v) = (-0.3-(-1), 3.2-3) \Rightarrow (0.7, 0.2)$

↑ Filtrarea texturii: Repeating

$(u,v) = (\min(\max(0, u_f), 1), \min(\max(0, v_f), 1)) \Rightarrow (0,1)$

↑ Filtrarea texturii: Clamping

° poți face clamping pe o coord și repeating pe cealaltă $\Rightarrow (0.7, 1)$, $(0, 0.2)$

(29) Eliminarea fețelor auto-obturate (BACK FACE CULLING) – produce substanțial nr. de poligoane rasterizate

→ efectuat pe GPU

- Algoritm: observatorul și obiectul sunt raportați la același sis. de coordonate, conturul fiecărei fețe a obiectului este orientat în sens trigonometric atunci când poliedrul este văzut din exterior, obs. situat în afara obiectului
- Fața vizibilă dacă $0 \leq \alpha < 90°$ și $0 < \cos(\alpha) \leq 1$; Condiția de viz: $Nu \cdot Ou > 0$

(30) STENCIL-BUFFER (la volume de umbră)

- Buffer-ul de marcaje (stencil-buffer) - în GPU cu buffer-ul imagine (frame buffer) și buffer-ul de adâncime (z-buffer), actualizat prin rasterizare

↳ DEPTH-PASS - stencil buffer-ul e modificat la reușita testului de adâncime

- calculul dacă obs. se află în mai multe vol. de umbră e obositor
- mai întâi desenăm fețele din față, apoi poligoanele din umbră, apoi fețele din spate

↳ DEPTH-FAIL - modificăm stencil buffer când testul de adâncime eșuează

- desenăm mai întâi fețele din spate, apoi poligoanele de umbră, apoi fețele din față.

(31) Maparea inversă : 1024×1024 , (0.4, 0.6) , GL_LINEAR?

$0.4 \cdot 1024 = 409.6$

$0.4 \cdot 1024 = 409.6 \rightarrow$ am făcut 1-0.6

$\Rightarrow (410,410), (409,410), (409,409), (410,409)$

(32) Fereastră - poartă
- fereastră : (0,0) - (1,1)
- poartă : (0,0) - (300,150)
- Transformările de scalare și translație? scalare uniformă și centrare în poartă

$$sx = \frac{x_{pmax} - x_{pmin}}{x_{fmax} - x_{fmin}} = 300 \qquad sy = 150$$

$$S = \min(sx, sy) = 150$$

$$tsx = \frac{x_{pmax} - x_{pmin} - s(x_{fmax} - x_{fmin})}{2}$$

$$tsx = \frac{300 - 150}{2} = 75$$

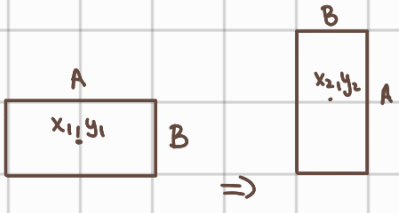
$$tsy = \frac{150 - 150}{2} = 0$$

⇒ ① Translatare fereastra în centru cu 75 și 0 (tsx, tsy)

② Scalez uniform cu 150,150 (S)

③ Translatare inversă cu -75 și 0 (-tsx, -tsy)

(33)



⇒ : Translație cu $(-x_1, -y_1)$ → ca să-l aducem în origine, Rotație față de origine cu 90°, Translație cu (x_2, y_2)

(34) Efectul de spot - în modelul de iluminare locală, sursa de lumină este considerată punctiformă. Lumina produsă de o sursă punctiformă este omnidirecțională

- efectul de spot simulează o sursă de lumină care produce un con de lumină, numai suprafețele aflate în conul de lumină primesc lumină de la sursă

- Conul de împrăștiere se calc. cu { poziția spotului (sursei de lumină); direcția spotului; împrăștierea luminii spotului (θspot sau cut-off)

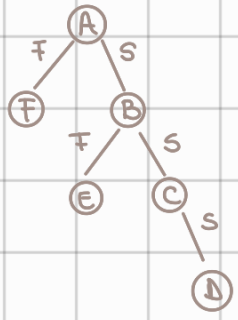
(35) (u,v) - (0.2, 3.3) - eșantionarea texturii

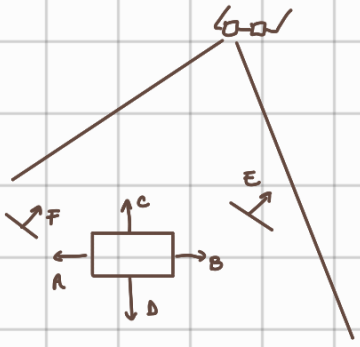
Repeating: (uf - [uf], vf - [vf]) ⇒ (0.2 - 0, 3.3 - 3) = (0.2, 0.3)

Clamping: (min(max(uf, 0), 1), min(max(vf, 0), 1)) = (0.2, 1)

(36) Alg BSP

a) ordine A, B, C, D, E, F





b) front to back

(37) Decuparea vectorilor în bandă grafică

→ în sis. grafic 2D — analitic - prin intersecția primitivelor cu frontiera zonei de decupare, înainte de rasterizare

— la momr rasterizării primitivelor, prin testul de apartenență a fiecărui fragment la dreptunghiul / poligonul de decupare

→ în sis. grafic 3D - decuparea se efectuează la frontiera volumului canonic, în coor de decupare (după trans. de proiecție, înainte de trans. în poarta de afișare)

* atributele noilor vârfuri se calc. prin interpolare.
* în bandă grafică { se realizează la frontiera vol. viz. canonic; înainte de coor. omogene, după coor. carteziene

(38) TRANSPARENȚA - maj. alg. de det. a viz. pot fi adapți pt. a îngloba transparența

- în OpenGL - transparența poate fi simulată prin amestecul dintre frag. curent și cea a pixelului în care se afișează, folosind modelul (R, G, B, A) - A-opacitat { 0 transparent; 1 opac
- se face cu GL_BLEND și glBlendFunc()
- operația de amestec: efectuată de procesul de rasterizare după testul de adâncime, înainte de actualizarea val. pixelului în buffer imagine

(39) **Modelarea reflexiei difuze** – dispersată regulat în toate direcțiile

– Legea lui Lambert

$Id = I_{sursă} * Kd * \cos(i)$; $0 \le i < \frac{\pi}{2}$

dacă $i > \frac{\pi}{2}$ ⇒ nu primește lumină de la sursă (în spate) — lumina e

(40) **În motor grafic**: Frustum culling și alg. BSP, Elim obiectelor, Alg. Pictorului

Pe GPU: Eliminarea fețelor auto-obturate ale obiectelor, testul de viz., decuparea primitivelor care intersectează vol. vizual

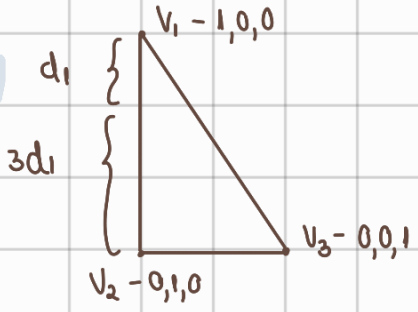
(41) **CEAȚĂ** – adaugă realism imaginii și ajută la eliminarea defectelor produse de decuparea la nivelul din spate al volumului vizual

- $0 \le \emptyset \le 1$ - factorul de ceață, proporțional cu distanța de la observator la punct ($\emptyset = 1$ opacitate)

– factorul de ceață liniar, acționează între 2 distanțe definite în sis. coord., măsurate pe direcția în care privește observatorul

- depinde de dist dintre observator și punctul în care se calculează cul. afectată de ceață

(42)



interpolare în modelul RGB

↳ ponderea 1/4

$c = t * C_2 + (1-t) \cdot C_1$

$c = \frac{1}{4} \cdot (0,1,0) + \frac{3}{4}(1,0,0) = (0.75, 0.25, 0)$

(43) **Generarea cercurilor în spațiul discret** – cercurile și elipsele nu sunt primitive grafice în OpenGL, deci nu pot fi trans. în bandă grafică, sunt primitive 2D în XOY

- pt. afișare sunt descompuse în primitive de tip vector sau punct ⌐ le putem asocia coord 2 și trans. în bandă grafică
  ⌙ mai eficient: Geometry Shader
  * folosind coon. carteziene ⊏ spațiere inegală, calcule complexe cu nr. reale
  * fol. coor. polare – avantajul calc. prin recurență: nu este necesar să se calc. sin și cos în fiecare iterație
  * fol. Alg. Bresenham

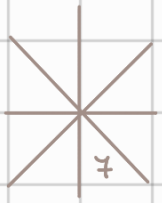
(44) **Alg. Bresenham generalizat** : Fie vectorul (8,11) → (14,3). Din ce octant face parte vectorul? Care este nr. de pixeli prin care este aprox. vectorul?

$dx = x_2 - x_1 = 14 - 8 = 6 > 0$

$dy = y_2 - y_1 = 3 - 11 = -8 < 0$

$dx < abs|dy|$

$\Rightarrow$ octantul 7



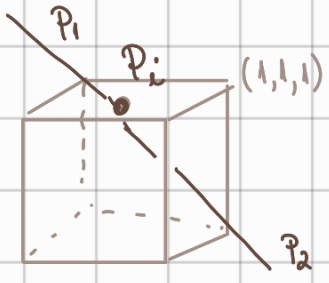
Nr. pixeli : $\max(abs(y_2 - y_1), abs(x_2 - x_1)) + 1$

$= 8 + 1 = 9$

(45) **Alg. Cohen-Sutherland - generalizare la 3D**

- e nevoie de 6 biți : $b_0 = 1$, stânga, $x < -1$ | $b_1 = 1$, dreapta, $x > 1$ | $b_2 = 1$, jos, $y < -1$ | $b_3 = 1$, sus, $y > 1$ | $b_4 = 1$, față, $z < -1$ | $b_5 = 1$, spate, $z > 1$

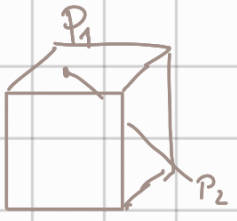
- dacă $cod(P_1) == 0$ && $cod(P_2) == 0$ $\Rightarrow$ ACCEPTAT
- dacă $cod(P_1) \,\&\, cod(P_2) \; != 0$ $\Rightarrow$ REJECTAT
- altfel se intersectează segmentul cu volumul



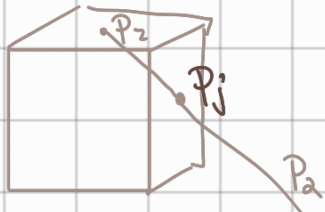
cod P_1 : 100100

cod P_2 : 011000

$\Rightarrow$ se intersectează în P_i $\Rightarrow$



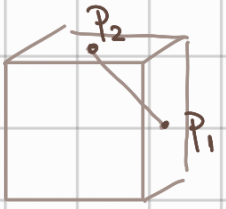
interschimbăm P_1 cu P_2 (P_1 e 000 000 după pasul ant)



$P_1 = 011000$

$P_2 = 000\,000$

$\Rightarrow$ se intersectează în P_j



$P_1 = 000\,000$

$P_2 = 000\,000$

$\Rightarrow$ ACCEPTAT